

Analysis: Ticket Trouble

Ticket Trouble: Analysis

There are two minor hurdles to deal with in this problem:

- Two tickets with the same numbers (e.g., both 1 2) can never both be in the same row, so they cannot both contribute to the maximum value that is the answer. Therefore, it is safe to eliminate or ignore one ticket from each such pair. You can sort the tickets so that duplicates end up next to each other, or use a data structure such as a set to eliminate them.
- A ticket might have the same number twice (e.g., 1 1). This can throw off an attempted solution that just counts how many times each number appears anywhere in any ticket.

A possible solution for the Small is to note that each of the F tickets might either be correct as is, or it might have its numbers reversed. You can try all 2^F possible scenarios and find the largest number of friends that ever end up together in the same row. The only tricky part is that duplicate tickets can cause illegal scenarios that put two friends in the same seat, so you must either check for these scenarios, or remove the duplicates in advance as mentioned earlier.

An even simpler solution that works for the Large is to determine the number of tickets that could possibly be in each row. For each ticket (ignoring one ticket from each duplicate pair), check the two numbers. If they are the same, add 1 to the count for that row number. If they are different, add 1 to the count for each of those row numbers. The maximum among these numbers is the answer. Note that in this solution, we do not need to explicitly consider any particular arrangement of the friends, and the solution would also work if the problem had asked about columns instead of rows.

Analysis: Understudies

Understudies: Analysis

Small dataset

There can be at most four roles and eight performers in a Small test case. We could write code to enumerate all of the different ways to pair up the performers, but this can be tricky to get right; however, the limits allow an even easier brute-force solution. It is simple to generate all possible permutations of performers using, for example, Python's `itertools.permutations`. For each permutation, we can pair the first performer in the list with the second, the third with the fourth, and so on, and then check the overall success probability in the way described in the sample case explanations. The maximum probability we encounter is the answer. This method will check equivalent casting decisions multiple times, but it doesn't matter; $8!$ is under 50 thousand, so your computer should not even break a sweat.

Large dataset

Brute force will not cut it for a musical with up to 40 roles — we must be giving *Cats* and *A Chorus Line* a run for their money! — so we need a more thoughtful strategy. Is it better to pair up reliable performers with other reliable performers, making some roles secure and others risky? Or should we pair our most reliable performers with our least reliable ones, spreading out the risk more evenly across roles? Or is the solution something more complex? Intuition can easily fool us when it comes to probability problems, so it's best to make a mathematical argument.

Suppose that we have paired up our performers in some arbitrary way. Let's consider two of the roles and the four performers. Without loss of generality, let's label the four performers A, B, C, and D, such that their probabilities of becoming unavailable follow the order $P_A \geq P_B \geq P_C \geq P_D$. Our key claim is that we will maximize these four performers' contribution to the overall probability of the show's success by pairing A with D and B with C, if they are not already paired in that way.

Let's prove that pairing A with D is better than pairing A with C. If we pair A with D, the probability that both of these roles will be successfully filled is as follows:

$$(1 - P_A P_D)(1 - P_B P_C) = 1 - P_A P_D - P_B P_C + P_A P_B P_C P_D.$$

If we instead pair A with C, the probability of success is:

$$(1 - P_A P_C)(1 - P_B P_D) = 1 - P_A P_C - P_B P_D + P_A P_B P_C P_D.$$

Subtracting the second quantity from the first, we get:

$$P_A P_C + P_B P_D - P_A P_D - P_B P_C = P_A(P_C - P_D) + P_B(P_D - P_C) = (P_A - P_B)(P_C - P_D).$$

Since we know that $P_A \geq P_B$, and $P_C \geq P_D$, both of the terms in the final expression above must be positive or zero, and so their product is also positive or zero. That means that we are at least as likely to succeed if we pair A with D as we are if we pair A with C. (The contributions from performers other than A, B, C, and D are identical across these two cases, so we don't need to

consider them here.) A similar argument shows that pairing A with B cannot possibly be better than pairing A with D.

So we have proven that in any arrangement, for any pair of roles, we should reassign the four performers as necessary to pair up the most reliable and least reliable of the four in one role, and the other two performers in the other. This implies that the most and least reliable performers in the entire cast should be assigned to the same role. (If they are in two different roles, just apply the argument above to that pair of roles.) Similarly, the second-most and second-least reliable performers should be in the same role, and so on. Only the rank orders of the performers' probabilities turn out to matter; the actual values are not important!

This makes our solution very simple: sort the probabilities and then pair up the extremes, and then the remaining extremes, and so on. This is $O(N \log N)$ — it would be linear if not for the sorting step — and it would easily work for values of N much larger than 40.

Analysis: Where Ya Gonna Call?

Where Ya Gonna Call?: Analysis

We can model the input as a [weighted undirected graph](#). The question to be answered is similar to finding the graph's [radius](#). In fact, we are asked to minimize the input graph's radius by possibly adding a single node inside an existing edge. This view is not only a succinct and accurate way to describe the problem, but also a first step towards a solution.

Small dataset

One solution for the Small relies on a simple fact: the result is always an integer multiple of $1/2$, and if the optimal location is inside an edge, it is always at a point that is at a distance from one of the incident nodes that is an integer multiple of $1/2$. This property is not hard to prove: if the location is in a building, all distances to it are integers. If not, let L be a location and B be a farthest building at distance D . Let B' be the farthest building among those whose minimum path approaches L from the other side of the edge as B 's minimum path, with distance D' . If there were no buildings whose minimal path approaches L from the other side, L would not be an optimal location, since we could move L towards B decreasing all distances. If $D' < D$, we could move the location slightly towards B and decrease the overall minimum, so $D' \leq D$, which together with $D' \geq D$ by definition implies $D' = D$. Notice that the fractional parts of both D and D' only depend on where L is located on the edge, because the rest of each distance comes from a distance between buildings, which is an integer. Therefore, since the fractional parts of D and D' are equal, the fractional parts of the edge on both sides of L are equal, and thus an integer multiple of $1/2$.

With this property in mind, and the limitation that the maximum edge length is 2 in the Small dataset, there are only 3 positions on each edge that can contain an optimal location (at distances 0.5, 1, and 1.5 from one end). For each of these, we can find the farthest building. To do that, we can use [Dijkstra's algorithm](#). We also have to consider the radius of the original graph (which represents choosing a location in a building), which we can do by running the same algorithm starting at each building. Then, we just take the minimum farthest distance from all of those options. The running time of this solution is $O(MB^4)$ where M is the maximum length of an edge. This is because we need to run Dijkstra's algorithm $2M-1$ times per edge for up to $O(B^2)$ edges and once per node for $O(B)$ nodes (a total of $O(MB^2)$ times), and each run takes $O(B^2)$ time.

Large dataset

The approach outlined for the Small dataset does not work for the Large, because the edges can be really long and thus the number of locations to try is too large, even when restricted to integer multiples of $1/2$.

What we can do to simplify the problem is to use [binary search](#). That is, write an algorithm to determine whether there is a location with farthest distance D or less. The statement is clearly false for some interval $[0, X)$ of values for D and true for $[X, \text{infinity})$, so we can simply binary search for X .

The simplification that we obtain is that we can now check for a fixed distance. We start by finding the distance between all pairs of buildings. We can either run Dijkstra's algorithm once per building as mentioned above, or use something simpler for all pairs like [Floyd-Warshall's](#).

We can then iterate over each edge to see whether there is a viable location within it. For a fixed edge, we iterate over each building and see where in the edge a location could be at a distance D or less from that building. To go to a point inside edge (I, J) from building K , there are two options: go from K to I and then move inside the edge, or go from K to J and then move inside the edge. Since we now know the distance from K to I and J , we can calculate the interval of positions within the edge that can be reached with distance D from each end I and J . Then, if those intervals cover the entire edge (by overlapping or because one of them is big enough), then any location inside (I, J) is reachable from K with a distance D or less. Otherwise, there is some interval of unreachable locations. We record such intervals for each building and then check whether the union of all those intervals covers all of (I, J) . If it does, no location inside (I, J) is viable. Otherwise, there is at least one that is.

To check if a given set of intervals fully covers another interval A there is a greedy algorithm: sort the intervals by starting point and process them while keeping a current covered upper bound U , which is initialized to the lower bound of A . While U is less than the upper bound of A , for each interval, if its lower bound L is greater than U , then there is an uncovered interval (U, L) and we are done. Otherwise, if the upper bound H of the current interval is greater than U , set $U := H$. If the iteration finishes due to U becoming larger than A 's upper bound, all of A is covered. Otherwise we either found a hole or there is one between the last value of U and the upper bound of A .

For this solution we run Floyd-Warshall's which takes $O(B^3)$ time and then run a procedure for each of $O(B^2)$ edges. This procedure iterates all B nodes within a binary search, which takes $O(\log BM)$ to converge (remember M is the maximum length of an edge, so BM is an upper bound on the output), which makes it take time $O(B^3 \log BM)$ overall.

Notice that since we are binary searching for X , precision is not an issue, as a bad decision due to precision would only give a slightly larger or slightly smaller result. Additionally, we could use the property of the result being a multiple of $1/2$ to do all calculations on integers by doubling all edges in the input and dividing by 2 at the very end.

A number of people tried to use [ternary search](#) to solve this problem. Ternary search assumes a convex or concave function, and the function in this case (the farthest distance for each point within an edge) is neither. The distance from each point to a single fixed building is indeed a concave function. However, the maximum of many concave functions is not a concave nor a convex function. Some ternary search implementations may succeed in the Small because of the really small number of critical points, which may all be tried even under the flawed assumption.

Analysis: Word Search

Word Search: Analysis

Small dataset

One surprisingly viable approach for the Small dataset is to create random grids of random sizes and count the numbers of I/O s in them until we happen to find a grid with the desired number. Even if we only try square grids with a random size and random contents, this is easily fast enough to pass. Other optimizations are possible — we can vary the frequencies of I , $/$, and O , or try to edit grids that are close — but not necessary.

Another possibility is to directly construct a grid with the desired number of I/O s. Better yet, we can construct one grid with at least 287 I/O s, and then use it in every test case, eliminating I/O s individually until we have the desired number. To ensure that I/O s do not interfere with each other, let's space them out in a field of O s like this:

```
I/OOI/OOI/OO...
OOOOOOOOOOOOO...
I/OOI/OOI/OO...
OOOOOOOOOOOOO...
I/OOI/OOI/OO...
...
```

If we extend this pattern to fill a 50 by 50 grid, we will have 25 rows with 12 I/O s each; $25 \times 12 = 300$, which is more than enough. We can eliminate an individual I/O by changing its $/$ into another O . With this grid setup, that change cannot possibly affect other existing I/O s or create new ones. Other similar strategies are possible.

Large dataset

The particular construction strategy above cannot produce enough I/O s when each of our grid dimensions is capped at 15. Nor will random generation help; we would be lucky to get a random 15 x 15 grid for $N = 80$, let alone 287. We need to pack as many I/O s into the grid as possible. Let's start with a single row; we can fit many I/O s in, overlapping forward ones with backward ones:

```
I/O/I/O/I...
```

If we stack these rows on top of each other, we will also form many diagonal I/O s:

```
I/O/I/O/I...
I/O/I/O/I...
I/O/I/O/I...
...
```

How many I/O s are in the 15 by 15 version of this grid? Let's count by looking only at the $/$ s. Any $/$ in the top or bottom row is part of exactly one I/O , running horizontally. Any other I/O is part of exactly three I/O s: one horizontal and two diagonal. There are 7 $/$ s in each row; the 14 in the top and bottom row contribute 14 I/O s, and the $13 \times 7 = 91$ in the other rows contribute

$91 \times 3 = 273$. That's a total of 287, which is conveniently exactly the maximum number of $\mathbb{I}/\mathbb{O}$ s we could be asked to produce. Note that we do not need to prove that this arrangement packs in as many $\mathbb{I}/\mathbb{O}$ s as possible; we only had to find a way to fit at least 287 in.

Now, in each test case, we only need to whittle the number of $\mathbb{I}/\mathbb{O}$ s down from 287 to $\mathbf{N}$. As we do this, we must be careful not to create any new $\mathbb{I}/\mathbb{O}$ s or destroy more $\mathbb{I}/\mathbb{O}$ s than we want to. One safe strategy is to change $/$ es into $\mathbb{O}$ s, as in the Small strategy above. Changing a $/$ in the top or bottom row eliminates one $\mathbb{I}/\mathbb{O}$, and changing any other $/$ eliminates three. We can reach any value of $\mathbf{N}$ by first removing as many "threes" as possible (without going below $\mathbf{N}$) and then removing as many "ones" as possible (without going below $\mathbf{N}$). For example, for $\mathbf{N} = 280$, we remove two "threes" and one "one". For $\mathbf{N} = 4$, we remove everything except for four of the "ones". Since there are only 288 possible test cases, we can even precompute answers to all of them before downloading the dataset (but this is not necessary).